Assignment 6

jQuery: selectors, effects and animations

Student: Makanow Madiyar

Group: IT 2515

Course: Web Technology

GitHub repository: https://github.com/coderkz07/Assignment6-WebTechnology

The project rebuilds the birthday party from the assignment picture. Task 1 answers twenty questions by searching the page. Task 2 adds a spotlight, animated guest details, room moves and sorting by arrival time.

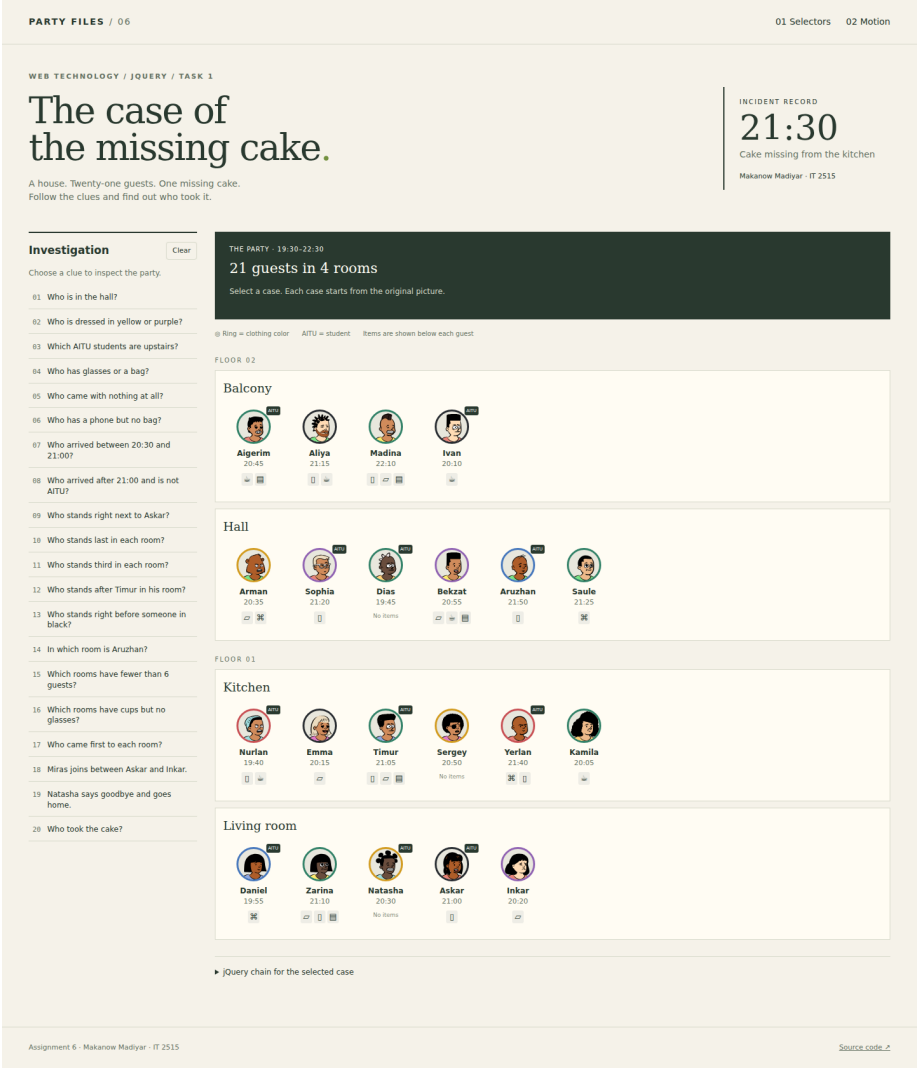


Figure 1. The party page, built from the room and guest array.

Task 1. Selector Detective

The array stores four rooms and 21 guests. Each room has an id, name, floor and guest list. Guest order follows the picture from left to right. Each guest stores a name, clothing color, arrival time, AITU status and items.

Floor	Room	Guests in picture order
2	Balcony	Aigerim, Aliya, Madina, Ivan
2	Hall	Arman, Sophia, Dias, Bekzat, Aruzhan, Saule
1	Kitchen	Nurlan, Emma, Timur, Sergey, Yerlan, Kamila
1	Living room	Daniel, Zarina, Natasha, Askar, Inkar

I checked the AITU badges against the picture: Arman has no badge and Natasha has one. These details affect case 3 and the neighbour checks.

```
{name: 'Arman', color: 'yellow', time: '20:35', aitu: false, items: ['bag', 'glasses'], order: 0}
```

buildPage() creates the floors, rooms and guest buttons with jQuery. makeCard() adds the local avatar, clothing-color ring, name, arrival time, item icons and AITU badge. Time is also stored as minutes after midnight for comparisons.

Information	DOM representation
Room and floor	.room id, data-room; .floor data-floor
Guest details	data-name, data-color, data-time, data-aitu, data-items
Time comparison	data-min, for example 20:35 = 1235
Items and student status	.has-phone, .has-glasses, .has-bag, .has-cup, .has-crumbs; .aitu

Each case uses one jQuery chain. The chain searches elements on the page instead of filtering PARTY. A click rebuilds the page, highlights the results, makes matching guests jump once with animate(), and displays the count and names. Cases 14-16 return rooms. Cases 18 and 19 demonstrate adding and removing a guest.

Case 1. Who is in the hall?

```
$('#hall .guest')
```

Result: 6 guest(s) - Arman, Sophia, Dias, Bekzat, Aruzhan, Saule.

The hall id limits the search to guest cards inside that room.

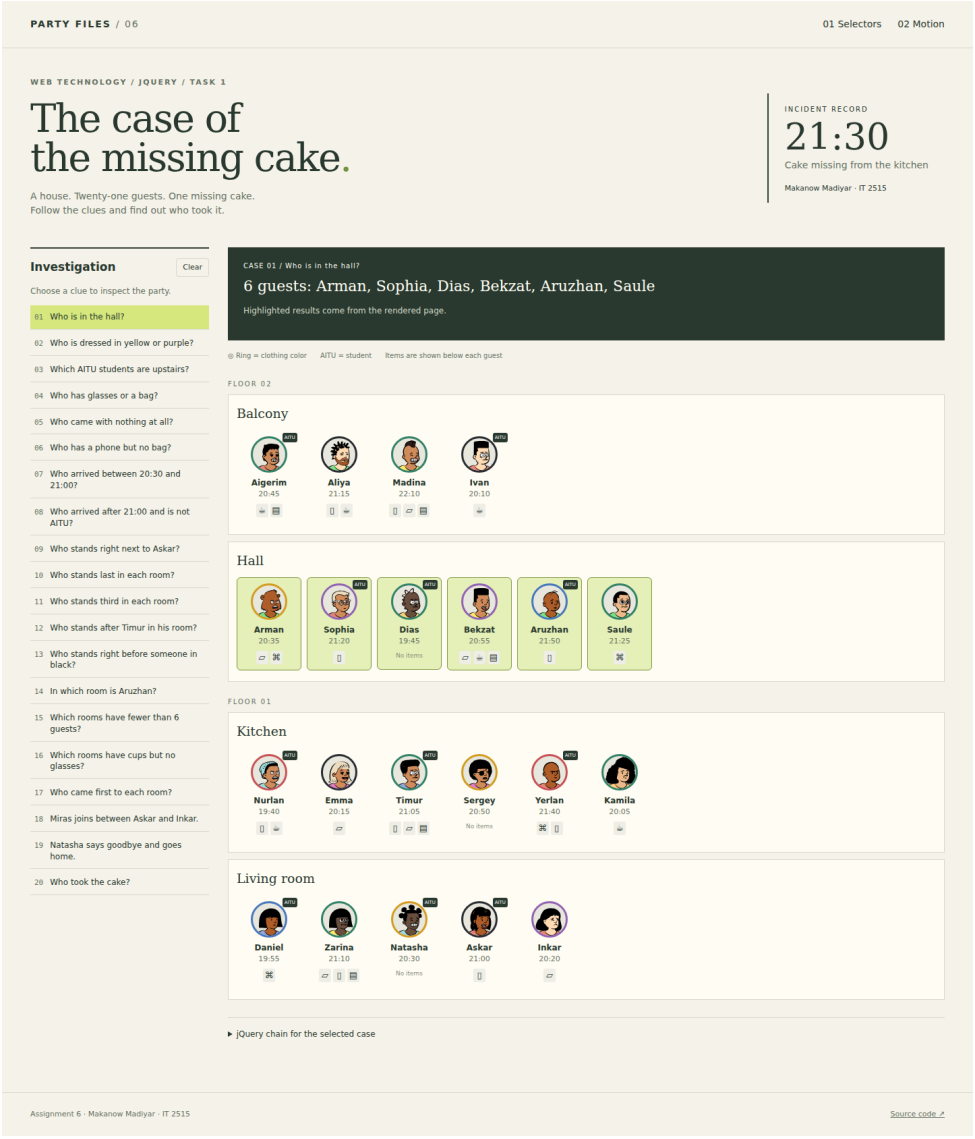


Figure 2. Result of case 1.

Case 2. Who is dressed in yellow or purple?

```
$('.guest[data-color="yellow"], .guest[data-color="purple"]')
```

Result: 6 guest(s) - Arman, Sophia, Bekzat, Sergey, Natasha, Inkar.

The comma combines two clothing-color attribute selectors.



Figure 3. Result of case 2.

Case 3. Which AITU students are upstairs?

```
$('.floor[data-floor="2"] .guest.aitu')
```

Result: 5 guest(s) - Aigerim, Ivan, Sophia, Dias, Aruzhan.

The floor attribute selects upstairs. The aitu class keeps student cards.



Figure 4. Result of case 3.

Case 4. Who has glasses or a bag?

```
$('.guest.has-glasses, .guest.has-bag')
```

Result: 10 guest(s) - Madina, Arman, Bekzat, Saule, Emma, Timur, Yerlan, Daniel, Zarina, Inkar.

The comma means a guest can have glasses, a bag, or both.



Figure 5. Result of case 4.

Case 5. Who came with nothing at all?

```
$('.guest:not([class*="has-"])')
```

Result: 3 guest(s) - Dias, Sergey, Natasha.

Guests with no items have no class containing has-. The not selector keeps them.



Figure 6. Result of case 5.

Case 6. Who has a phone but no bag?

```
$('.guest.has-phone:not(.has-bag)')
```

Result: 6 guest(s) - Aliya, Sophia, Aruzhan, Nurlan, Yerlan, Askar.

The guest must have the phone class and must not have the bag class.



Figure 7. Result of case 6.

Case 7. Who arrived between 20:30 and 21:00?

```
$('.guest').filter(function () { return +$(this).attr('data-min') >= 1230 && +$(this).attr('data-min') <= 1260; })
```

Result: 6 guest(s) - Aigerim, Arman, Bekzat, Sergey, Natasha, Askar.

The filter compares minutes against 1230 and 1260. Both boundaries are included.



Figure 8. Result of case 7.

Case 8. Who arrived after 21:00 and is not AITU?

```
$('.guest:not(.aitu)').filter(function () { return +$(this).attr('data-min') > 1260; })
```

Result: 4 guest(s) - Aliya, Madina, Saule, Zarina.

The first selector excludes AITU students. The filter keeps arrivals later than 21:00.



Figure 9. Result of case 8.

Case 9. Who stands right next to Askar?

```
$('.guest[data-name="Askar"]').prev('.guest').add('.guest[data-name="Askar"] + .guest')
```

Result: 2 guest(s) - Natasha, Inkar.

prev() finds the left neighbour. The adjacent sibling selector finds the right neighbour.



Figure 10. Result of case 9.

Case 10. Who stands last in each room?

```
$('.guests > .guest:last-child')
```

Result: 4 guest(s) - Ivan, Saule, Kamila, Inkar.

last-child selects the last guest inside each room container.



Figure 11. Result of case 10.

Case 11. Who stands third in each room?

```
$('.guests > .guest:nth-child(3)')
```

Result: 4 guest(s) - Madina, Dias, Timur, Natasha.

nth-child(3) selects the third guest in every room, counting from one.



Figure 12. Result of case 11.

Case 12. Who stands after Timur in his room?

```
$('.guest[data-name="Timur"]').nextAll('.guest')
```

Result: 3 guest(s) - Sergey, Yerlan, Kamila.

nextAll() returns Timur's following guest siblings in the same room.



Figure 13. Result of case 12.

Case 13. Who stands right before someone in black?

```
$('.guest[data-color="black"]').prev('.guest')
```

Result: 4 guest(s) - Aigerim, Madina, Nurlan, Natasha.

The selector finds black clothing. prev() returns the guest immediately before each match. Emma is first in the kitchen, so she has no previous guest.



Figure 14. Result of case 13.

Case 14. In which room is Aruzhan?

```
$('.guest[data-name="Aruzhan"]').closest('.room')
```

Result: 1 room(s) - Hall.

closest() travels up from Aruzhan to the room section containing her.

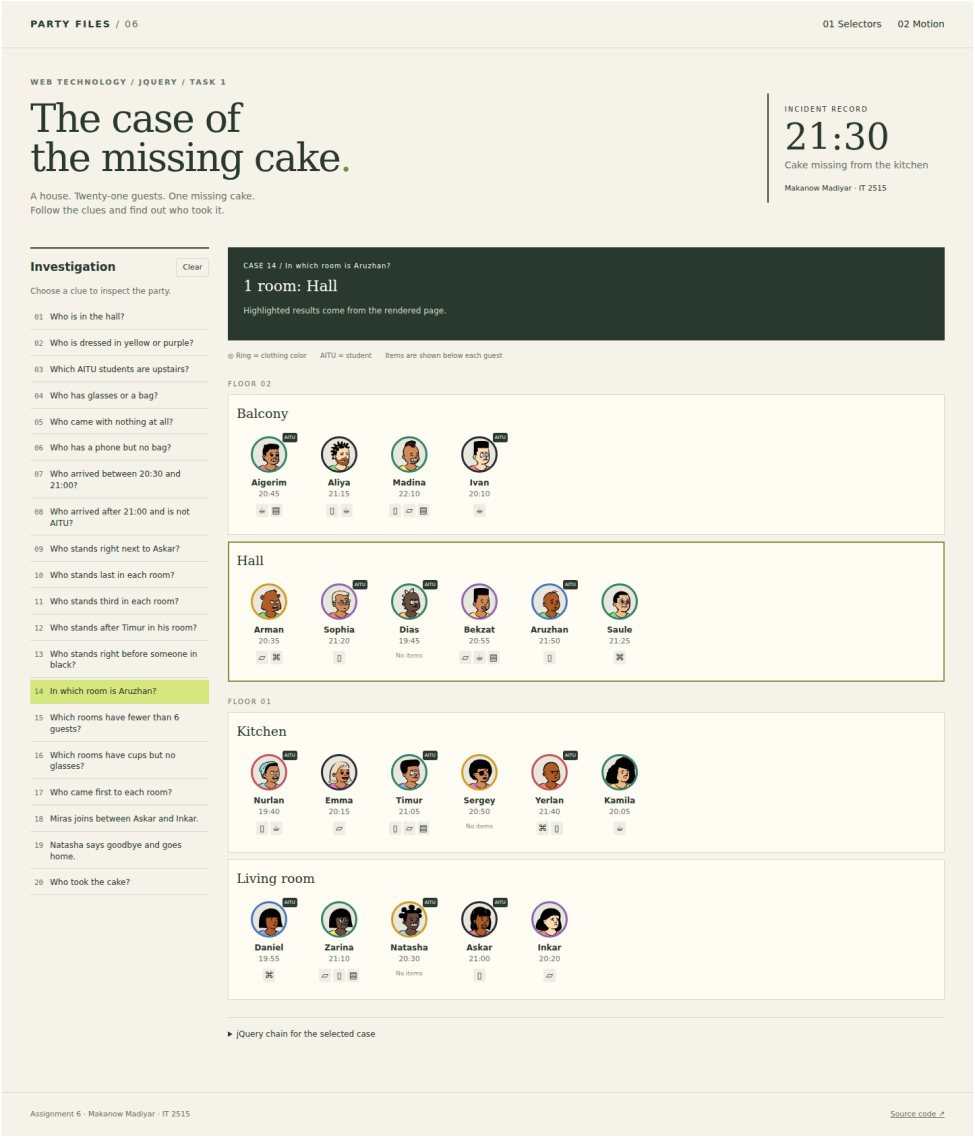


Figure 15. Result of case 14.

Case 15. Which rooms have fewer than 6 guests?

```
$('.room').filter(function () { return $(this).find('.guest').length < 6; })
```

Result: 2 room(s) - Balcony, Living room.

The filter counts guest cards in each room and keeps counts below six.

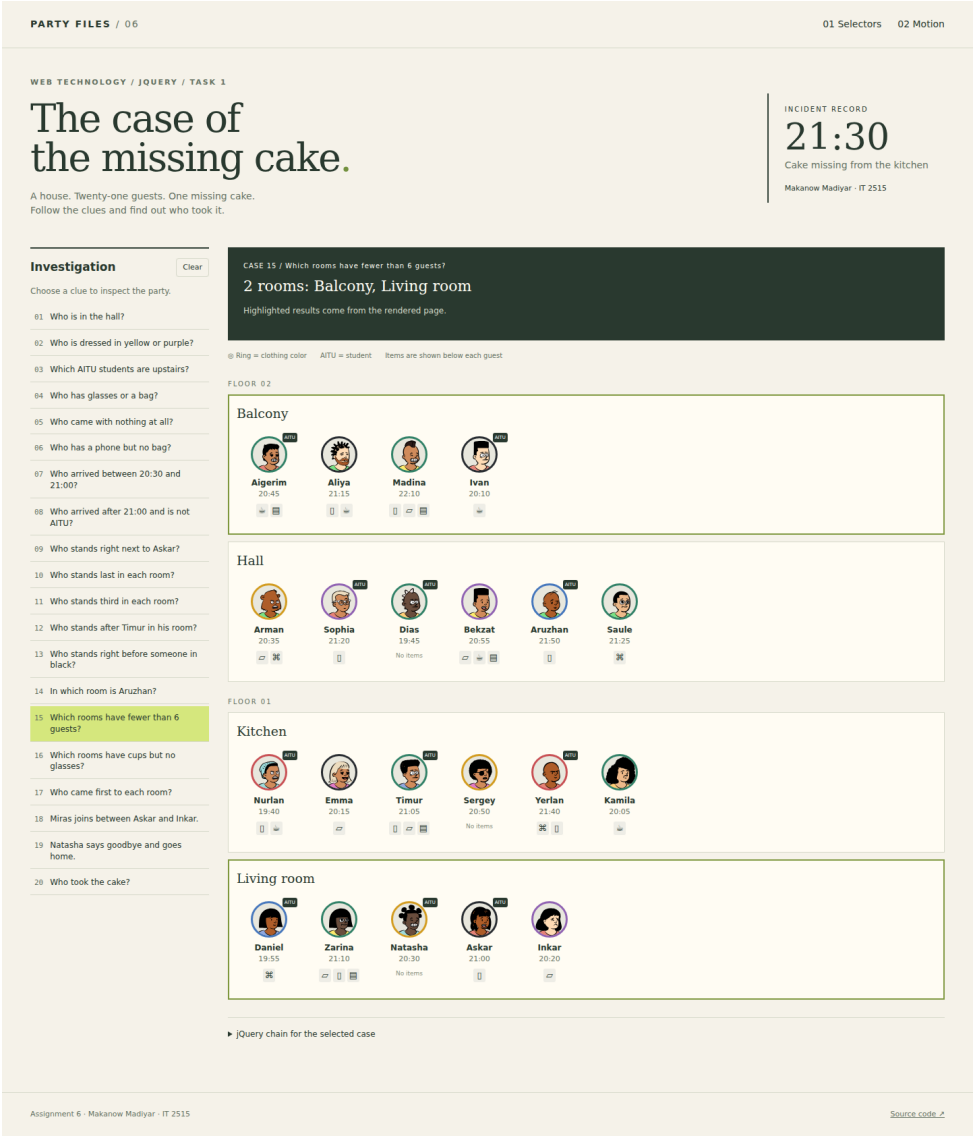


Figure 16. Result of case 15.

Case 16. Which rooms have cups but no glasses?

```
$('.room:has(.has-cup):not(:has(.has-glasses))')
```

Result: 1 room(s) - Balcony.

has() requires a cup in the room. The nested not selector excludes rooms containing glasses.

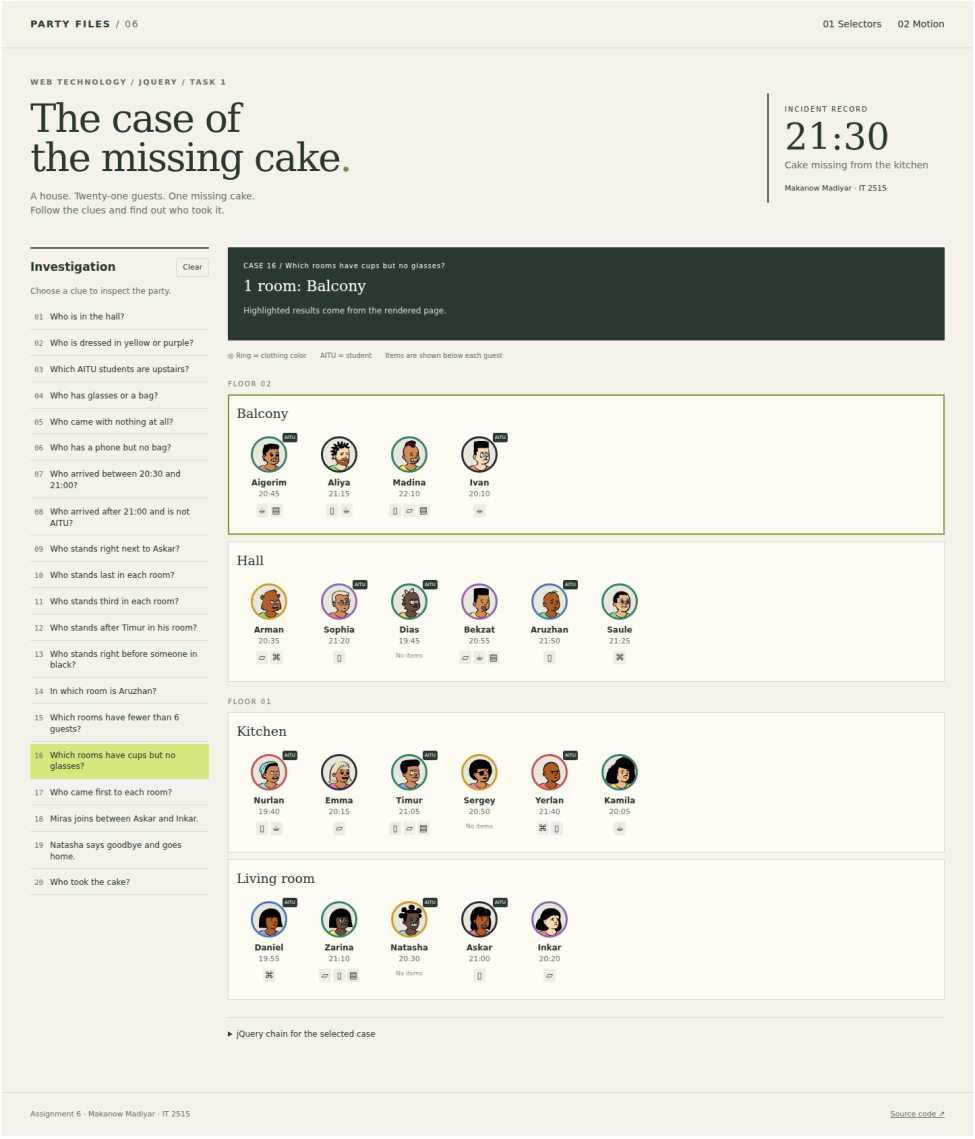


Figure 17. Result of case 16.

Case 17. Who came first to each room?

```
$('.guest').filter(function () { var time = +$(this).attr('data-min'); return
$(this).siblings('.guest').filter(function () { return +$(this).attr('data-min') < time; }).length
=== 0; })
```

Result: 4 guest(s) - Ivan, Dias, Nurlan, Daniel.

A guest arrived first if no sibling has a smaller arrival time. This checks time rather than position.

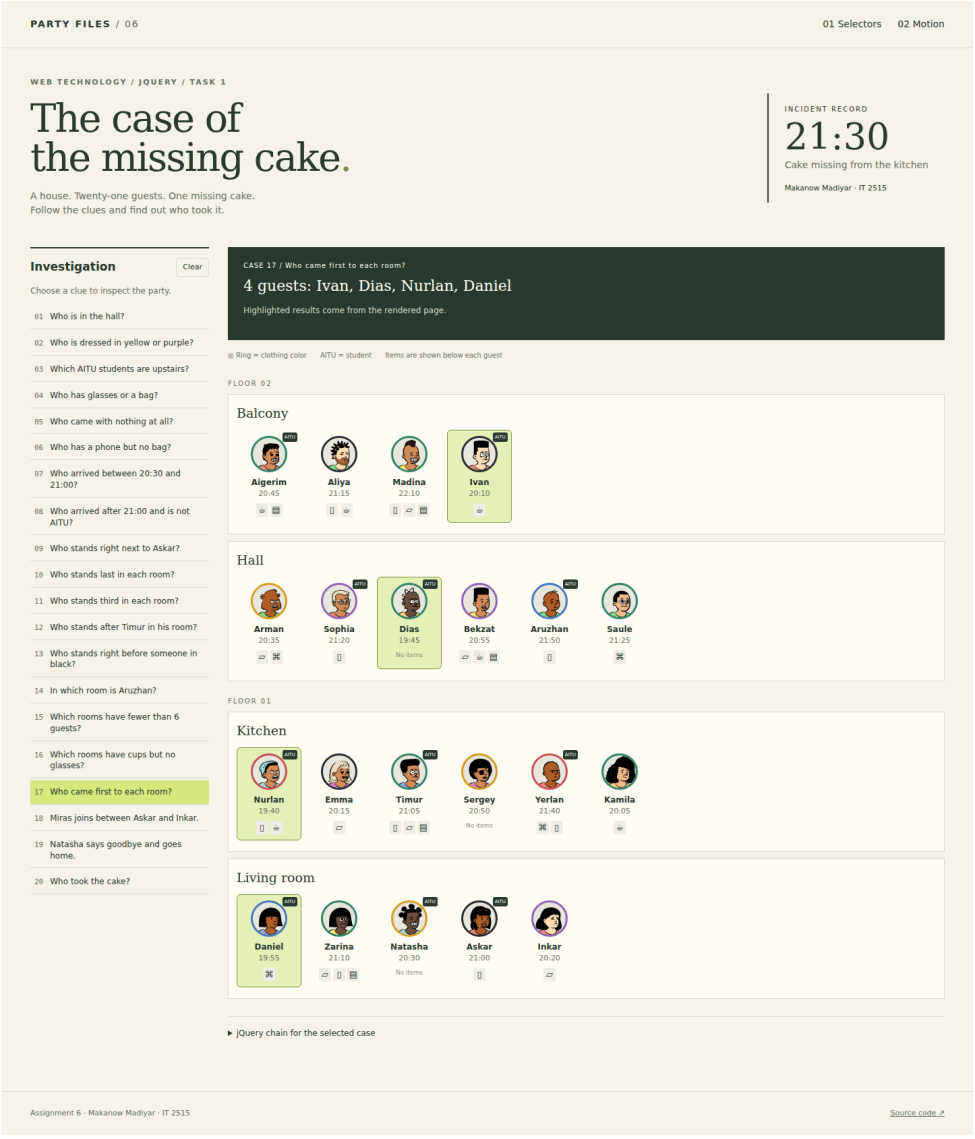


Figure 18. Result of case 17.

Case 18. Miras joins between Askar and Inkar.

```
makeCard({name:'Miras',color:'blue',time:'21:20',aitu:true,items:['phone']}).hide().insertAfter('.gu
est[data-name="Askar"]').fadeIn(650)
```

Result: 1 guest(s) - Miras.

makeCard() creates Miras. hide(), insertAfter() and fadeIn() place him after Askar and reveal him smoothly.

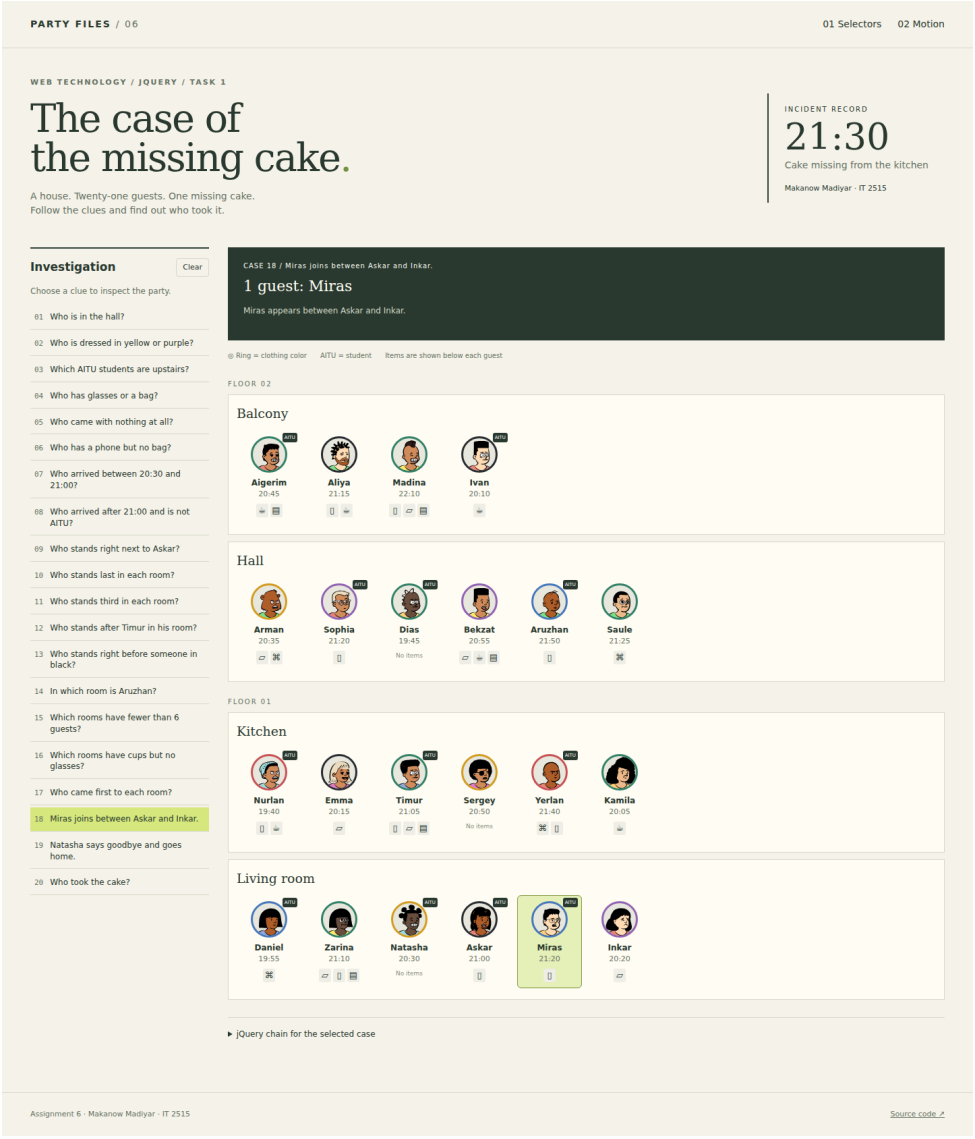


Figure 19. Result of case 18.

Case 19. Natasha says goodbye and goes home.

```
$('.guest[data-name="Natasha"]').fadeOut(650, function () { $(this).remove(); })
```

Result: 1 guest(s) - Natasha.

fadeOut() hides Natasha. Its completion callback removes the card from the DOM.

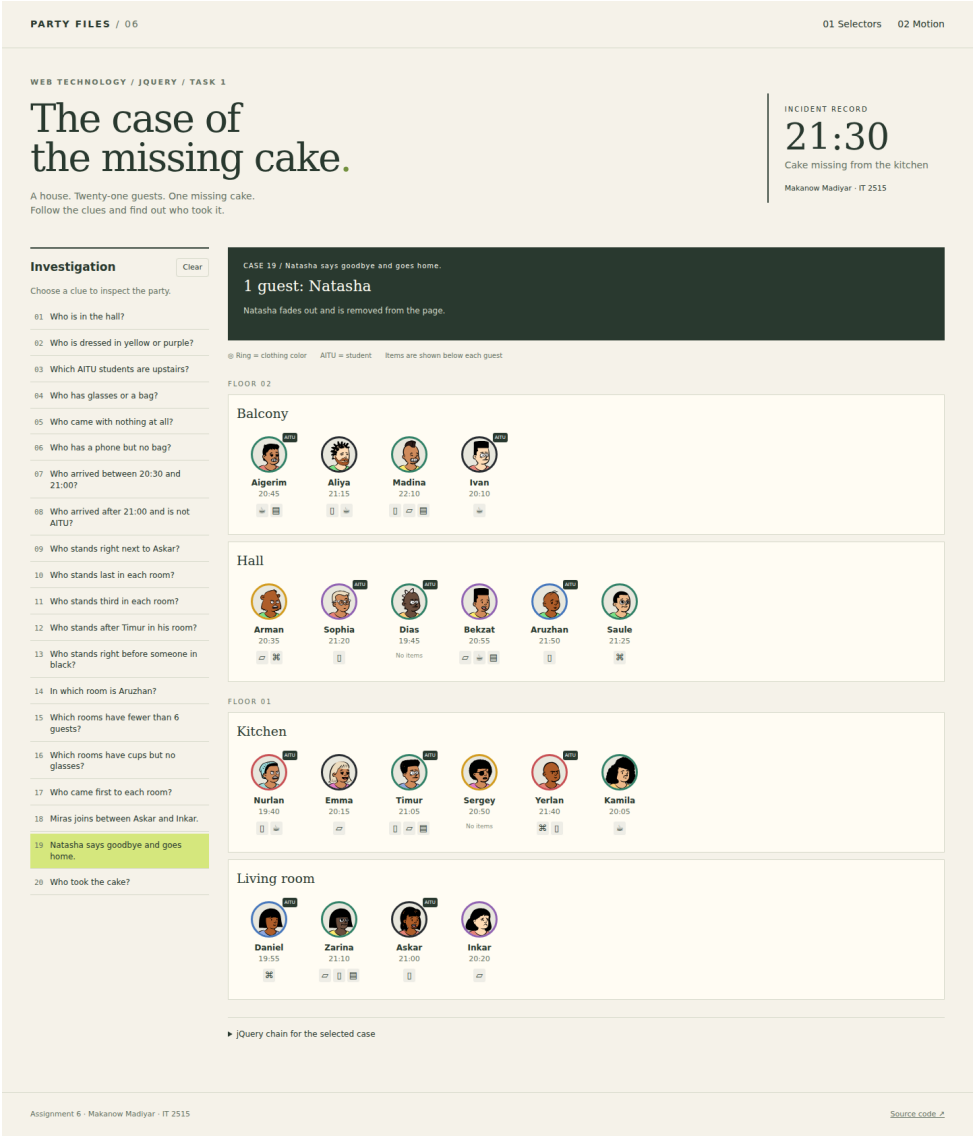


Figure 20. Result of case 19.

Case 20. Who took the cake?

```
$('.floor[data-floor="1"] .guest.has-crumbs:not(.has-cup)').filter(function () { return
+$(this).attr('data-min') > 1260 && +$(this).attr('data-min') < 1290; }).filter(function () { return
 $(this).prev('.guest').add($(this).next('.guest')).filter('.aitu').length > 0; })
```

Result: 1 guest(s) - Zarina.

The chain checks the downstairs floor, crumbs, absence of a cup, arrival after 21:00 and before 21:30, and an immediate AITU neighbour.



Figure 21. Result of case 20.

Task 2. The party in motion

Task 2 has its own copy of app.js and continues Task 1 with effects.js. The two tasks share the data, selector chains, stylesheet and avatars. All interactive behavior uses jQuery; there are no inline onclick handlers, CSS transitions or CSS keyframe animations.

1. Spotlight

The matched guests stay bright. All other guests animate to opacity 0.2 in 300 milliseconds. When a case returns rooms, the guests inside those rooms stay bright. Clear rebuilds the current room state with full opacity. A named spotlight queue prevents this fade from interrupting a sorting animation.

```
$('.guest').stop('spotlight', true)
  .animate({opacity: 0.2}, {duration: 300, queue: 'spotlight'})
  .dequeue('spotlight');
```

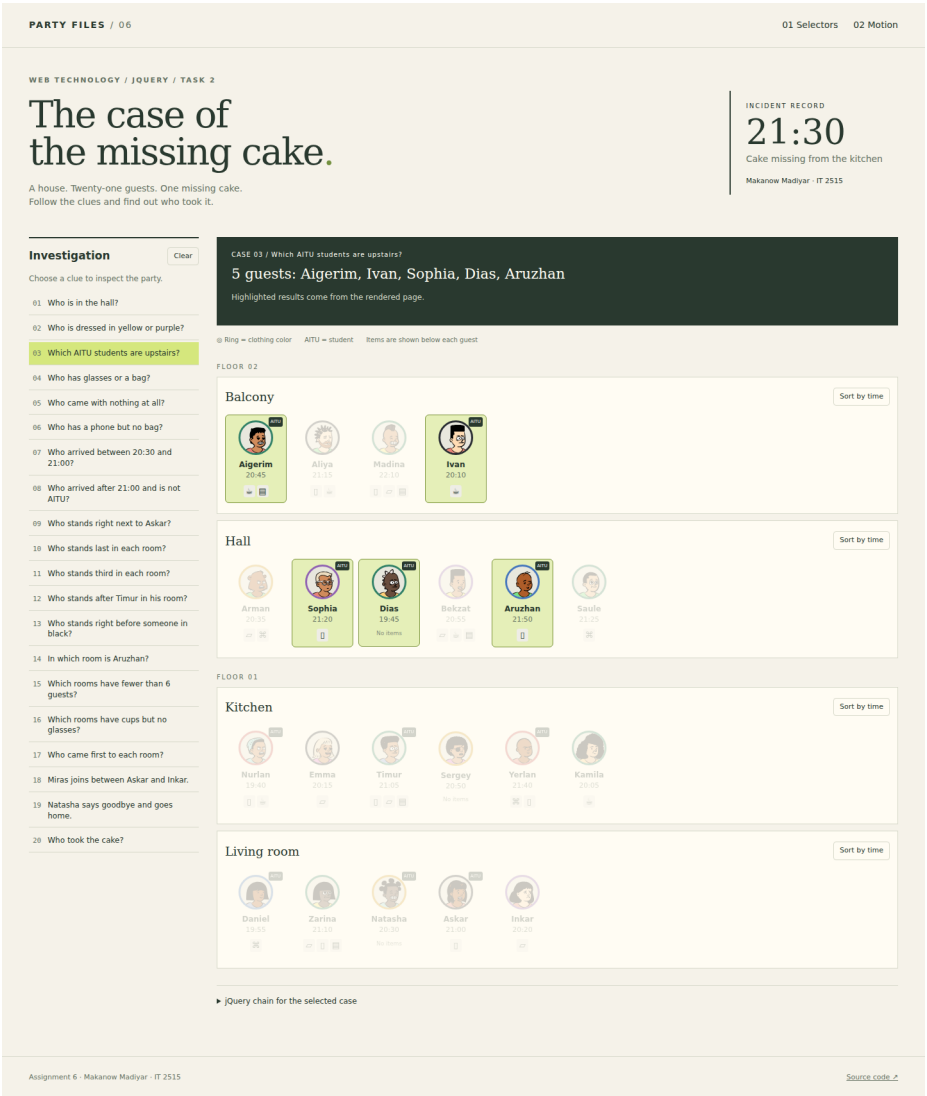


Figure 22. Case 3 keeps the five upstairs AITU students bright.

2. Guest details

Clicking a guest opens one shared detail panel. Its content describes the arrival time, clothing, student status and items in words. offset() finds the avatar position. The panel starts there at 58 by 58 pixels and animate() grows it to the measured content size. Escape, the close button and a click outside shrink the panel back to the avatar.

A version counter handles rapid guest clicks. Only the latest requested guest opens after the previous panel closes. stop(true, false) cancels an unfinished panel animation before starting the next one.

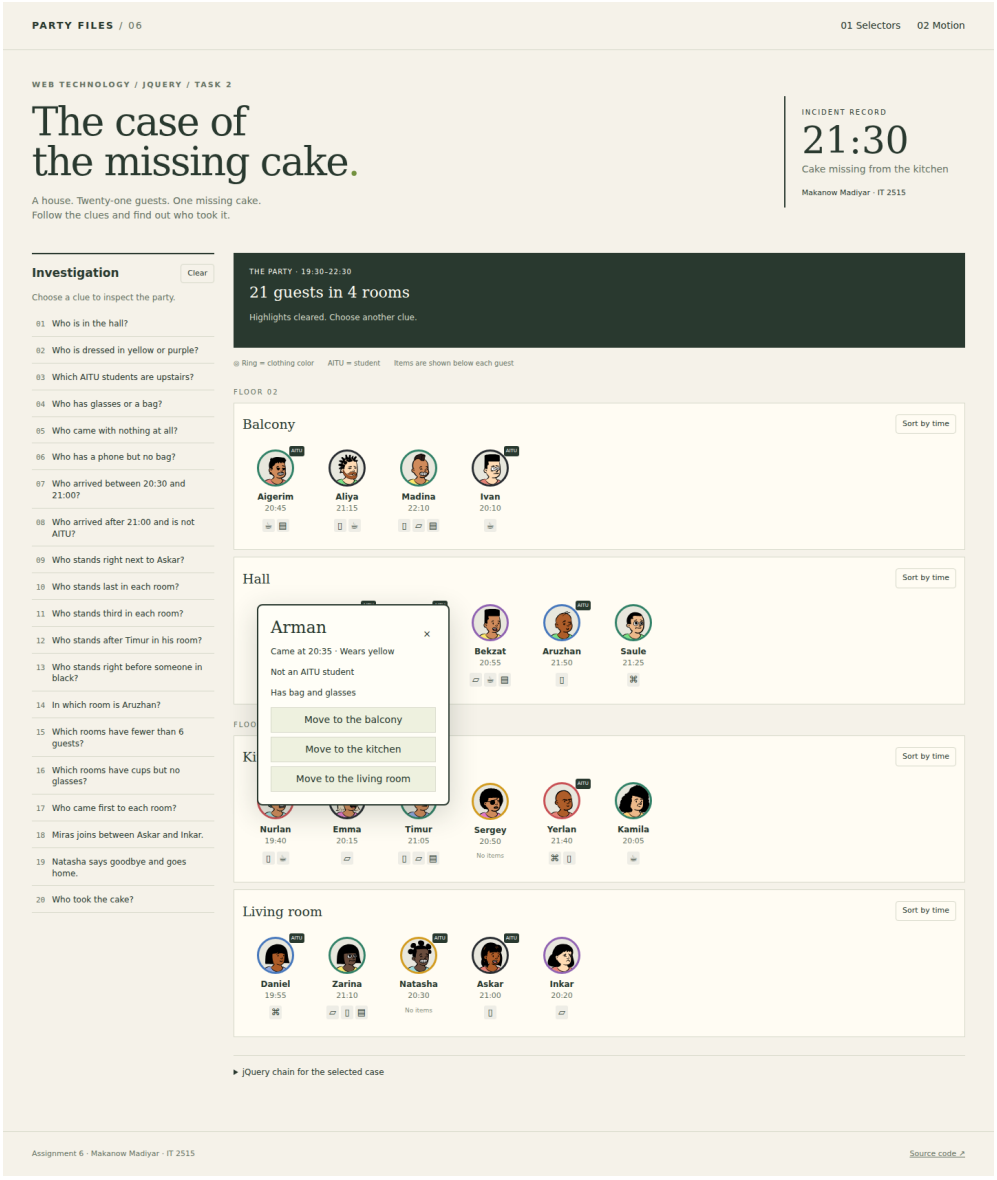


Figure 23. Arman's detail panel includes the correct student status and room buttons.

3. Moving a guest

A room button first updates PARTY: it removes the guest from the old room and appends the guest to the destination room. A clone flies between the old and new offsets over 700 milliseconds. The old card collapses while a destination slot opens. The clone is removed when the real card replaces the slot.

The destination returns to picture-order mode so the moved guest stays at its end. Before a new move or case, unfinished moves settle and the page rebuilds from PARTY. This avoids duplicate guests and leftover clones. After moving Arman to the balcony, case 1 finds five hall guests.

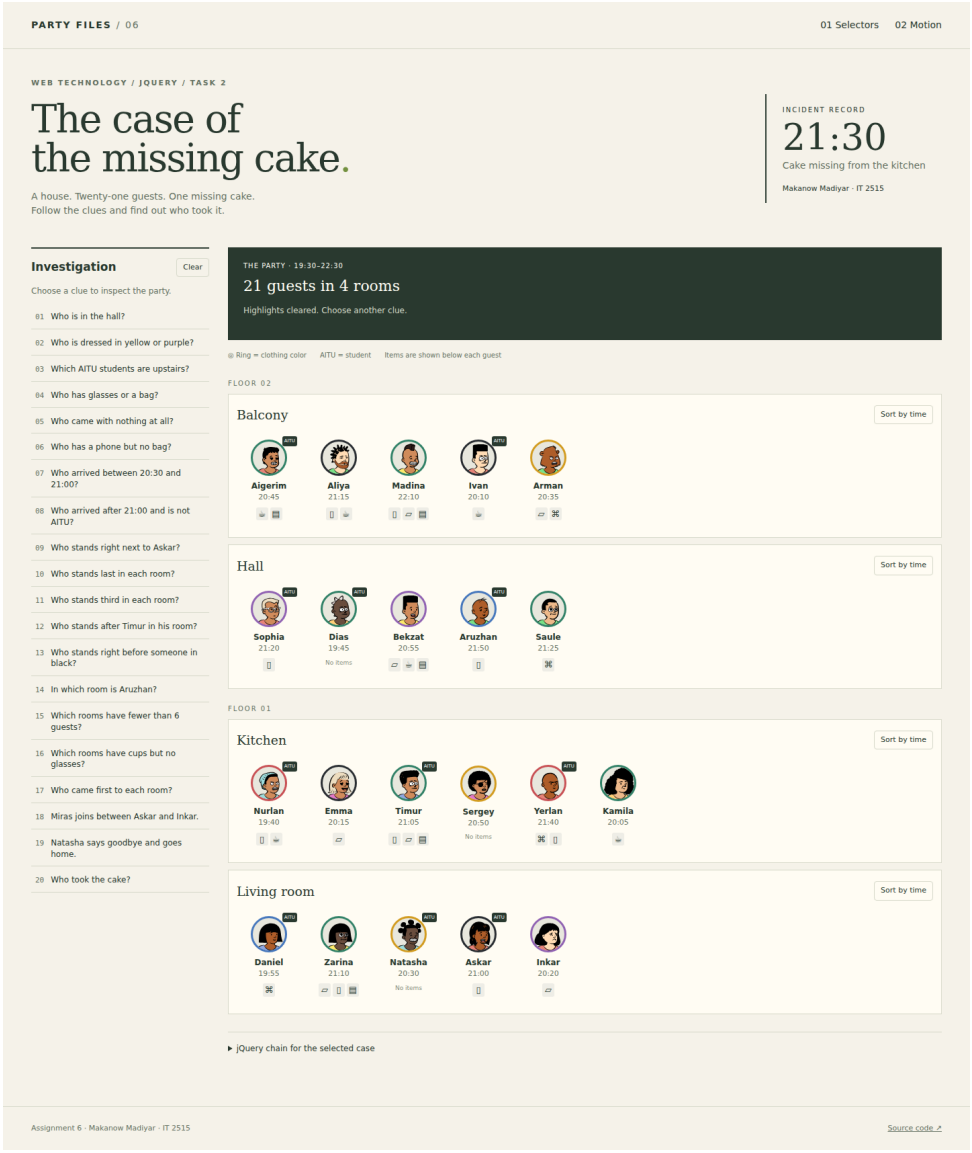


Figure 24. Arman is last on the balcony and no longer appears in the hall.

4. Sorting and fast clicks

The room button toggles between arrival time and picture order. The original order is stored on each guest. Before reordering, the code saves the visible position of every card. After append() changes the DOM order, the cards receive offsets that place them at their old positions. animate() moves those offsets to zero over 600 milliseconds.

The sort state changes immediately. A new click stops the previous sort queue and starts from the current visible positions. Five rapid clicks finish in arrival order; two clicks restore picture order. Guests who move into a room receive an order value at the end of its list.

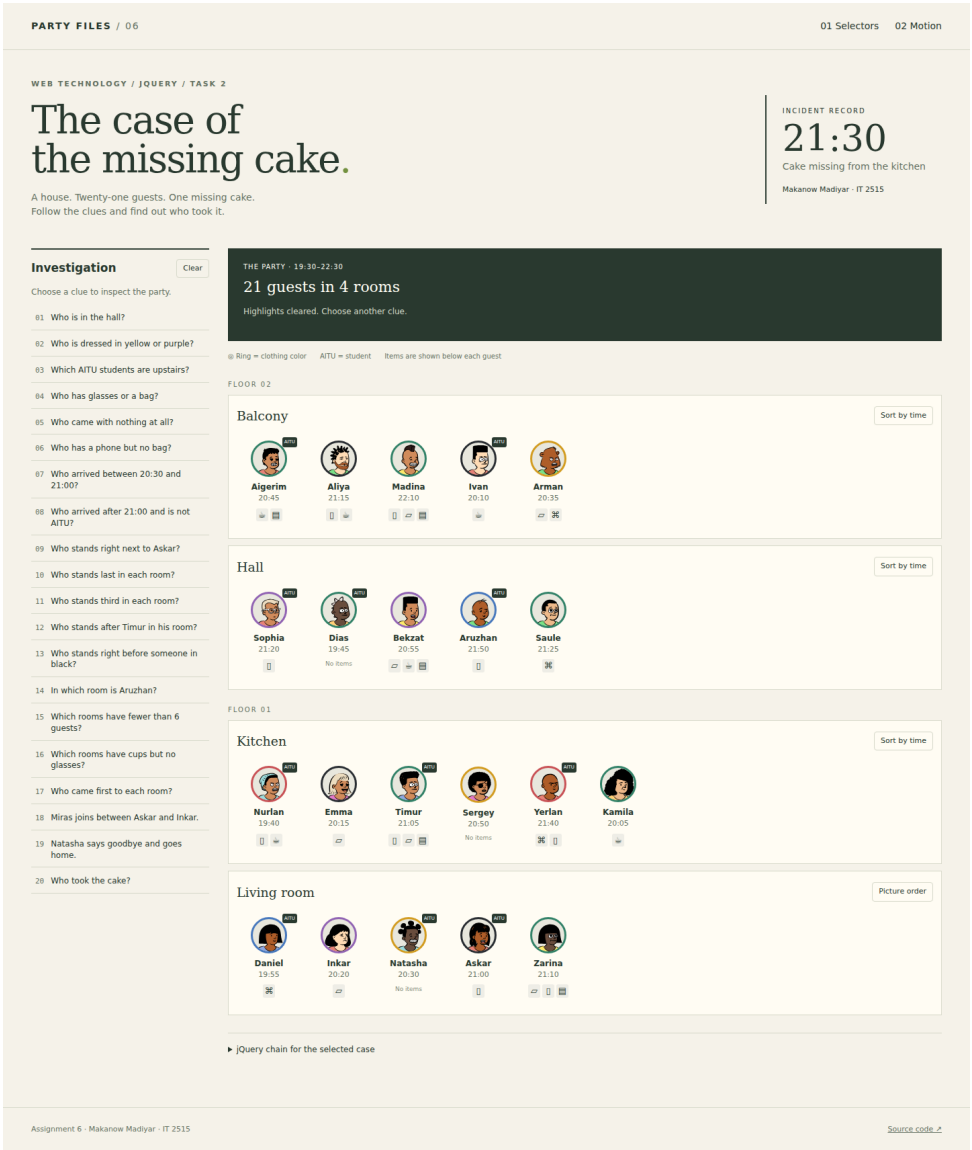


Figure 25. Living room ordered by arrival: Daniel, Inkar, Natasha, Askar, Zarina.

Verification

The following checks were run in a local Chromium browser against the completed pages. Microsoft Edge and the public deployment have not been verified because the Windows browser connection was unavailable.

Check	Observed result
Spotlight	Passed - 5 bright; 16 pale
Clear	Passed
Guest details	Passed
Escape	Passed
Move then case 1	Passed
Sort by arrival	Passed
Restore picture order	Passed
Five rapid sort clicks	Passed
Rapid guest clicks	Passed
Rapid moves	Passed
Unique guests	Passed

All 20 selector results were checked. Case 3 returns five guests. Case 13 returns Aigerim, Madina, Nurlan and Natasha. The result of case 20 is Zarina.

Conclusion

Selectors work best when the page stores clear classes and data attributes. Attribute selectors handle clothing and floor information, while traversal methods answer questions about neighbours and rooms. The cake clues identify Zarina: she was downstairs, arrived at 21:10, had crumbs and no cup, and stood next to AITU students. Timur meets the time and item clues, but neither of his neighbours is an AITU student.

The animation work showed why data updates and visual effects need separate responsibilities. PARTY records room membership, while jQuery animates the change. Stopping queues and keeping only the latest guest-card request makes fast clicks finish in the expected state.

Source code repository: https://github.com/coderkz07/Assignment6-WebTechnology

Libraries and artwork: jQuery 3.7.1 (MIT), https://jquery.com/; DiceBear Open Peeps avatars (CC0), https://www.dicebear.com/styles/open-peeps/. Avatars are saved locally.